SPM Modular Project: Modulo1

SPM course a.y. 25/26

Partitioned Hash Join with Duplicates

Project Overview

In this project you will implement a partitioned hash join with duplicates, a common building block in data analytics. In an equi-join, you need to find all pairs of records $r \in R, s \in S$ such that $r.key = s.key$, even when the same key appears multiple times in either relation.

A standard way to parallelize this problem is to decompose it into smaller independent subproblems. This can be done by partitioning the input dataset. Each input record is assigned to a partition based on its key, typically using a hash function. All records whose keys map to the same partition are grouped together. After partitioning, you can process each partition independently, building a compact lookup structure for the R-partition and probing it with the S-partition.

The first operation you perform in this parallelization pipeline, and one you may repeat in later phases, is mapping each 64-bit key to a partition identifier in $[0, P)$. This "key to partition id" mapping is a streaming kernel applied a large number of keys. Its performance often has an important impact on end-to-end time because it is executed over the full input size and it sits on the critical path before any per-partition work can start.

Module 1: Vectorization of the Partition Mapping Kernel

In the first module you focus on designing, implementing, and analyzing a high-performance implementation of the keys mapping. You will explore compiler auto-vectorization, manual SIMD intrinsics, and optionally GPU offload. The goal is not only to obtain speedups, but also to understand what limits performance.

Given an input array of N $keys$ (uint64_t, 64-bit unsigned integer) and a number of partitions P (usually a power of two), compute for each key a partition identifier in $[0, P)$. The mapping must be deterministic and must distribute keys reasonably across partitions for typical inputs. Input $keys$ are generated deterministically from a seed, so that the output is reproducible. The output is an array $part_id$ of length N. For each input key $keys[i]$, $part_id[i] \in [0, P)$ is the partition index assigned to that key.

Tasks

1. Provide one C++ implementation without intrinsics (plain version). Produce two binaries from the same source: one compiled with auto-vectorization disabled (baseline) and one with auto-vectorization enabled. You must provide evidence that GCC vectorized the hot loop(s) using compiler reports.

2. Provide a version using intrinsics AVX2 available on node09 (gpu-shared and gpu-exclusive partitions) and on the front-end node of the spmcluster. It must produce identical output to your plain C++ version.

3. (Optional) Provide a CUDA implementation that computes the same output array on GPU. If you do this extra task, you must separately report kernel-only time and host-device transfer time, and you must verify correctness against the CPU reference.

Deliverables

Send the teacher an email with subject "SPM Modulo1" which has attached a zip file with the name Modulo1_NameSurname.zip **by 31 March** (soft deadline). The zip file must contain:

- Source code and build instructions (Makefile or explicit compile commands).
- README containing instructions on how to run the tests and perform correctness checks.

- A short PDF report (max 4 pages) that includes:
    - Your design choices for the mapping function and for P (and why).
    - Evidence of GCC auto-vectorization (what flags you used and what the compiler reported – a short excerpt of the GCC report).
    - A brief explanation of your intrinsics strategy
    - Performance results obtained on node09 and a discussion of what limits performance (compute vs memory vs overhead).
    - If you implement the CUDA optional part, include correctness verification and a breakdown of time components.

Correctness and test requirements

You must implement a verification strategy that can detect mismatches without printing the full output array (for example a checksum, or hash over the output). For at least one small input size, it must be possible to perform an element-by-element comparison between implementations.

You must benchmark some value for N and at least one N that is "large enough" to obtain stable timings (order of magnitude: tens of millions). Your PDF report should report execution time, throughput (elements/s), and speedup of each optimized version over the plain C++ version. Use a repeatable methodology, for example multiple repetitions and reporting a median time and the standard deviation

<u>Suggested experiments (not mandatory):</u>

- Sweep P to study its effect on performance and distribution.
- Sweep key_space (or duplicate rate) to study sensitivity to input distribution.

<u>Notes</u>

You are expected to make reasonable engineering choices and justify them. There is no single correct mapping function or single correct implementation approach, provided your outputs are correct and your analysis is convincing. Avoid changing semantics between versions. If you explore alternative mappings, treat them as separate experiments and clearly label them.